

Amazon ML Challenge 2026

Business Entity Resolution: Complete Step-by-Step Guide

What every file is, what to do with it, and how to go from raw data to a scored submission

Challenge window: 25 Sep 2026, 12:00 AM IST to 27 Sep 2026, 11:59 PM IST. **Submissions:** max 5 per day on the leaderboard portal. **Metric:** macro-averaged F0.5 per Source 1 entity (precision counts 2x recall). **Final ranking:** private leaderboard.

Contents

1. The problem in plain words
2. Map of every file you received (and what to do with it)
3. What the data looks like (real examples and statistics)
4. How you are scored
5. Project folder you should create
6. Step-by-step pipeline (with code)
7. Producing the two output files
8. Validating and uploading
9. Final submission zip
10. Rules that get you rejected or disqualified
11. 3-day schedule
12. Final checklist

1. The problem in plain words

You receive business records from **three independent sources**. Each record has only a name, an address and a country. The same real business appears in different sources with typos, abbreviations, missing fields, different scripts (Hindi vs English), website-style names, or a completely different trade name. There is no shared ID.

Your job: for every record in **Source 1** (a clean, deduplicated reference list), output the list of all records in **Source 2 and Source 3** that refer to the same real business. The list can be empty (a *singleton*), have one ID, or have many IDs.

This task is called **Entity Resolution (ER)**. The standard solution has two stages:

- **Blocking / candidate generation:** cheaply shortlist, for each Source 1 record, a small number of plausible Source 2/3 records (comparing all pairs is impossible: $1.7M \times 10M = 17$ trillion pairs).
- **Matching:** compute similarity features for each shortlisted pair and use an ML classifier to decide match / no-match. Then pick a threshold that maximises F0.5.

2. Map of every file you received

Important: the folder **D:__MACOSX** contains only macOS metadata junk (files starting with '.'). Ignore it completely. The real data is in **D:\student_resource**. The two PDFs are also in **D:__MACOSX**.

File / folder	What it is	What you do with it
6ab5628d5a817_amazon_ml_challenge_problem_statement.pdf	Official problem statement	Read once fully. Same content as student_resource/README.md (easier to read).
6ab56657b4f1a_guidelines_and_key_instructions_...2026.pdf	Rules of the event: dates, 5 submissions/day, no multiple logins, query form	Read once. Remember: desktop only, one login per person, keep version history of every submission.
student_resource/README.md	Problem statement in Markdown	Your reference while coding (format rules, metric, zip structure).
student_resource/Documentation_template.md	Methodology write-up template	Fill it in at the end (Day 3). Goes into the final zip at the top level, not renamed.
student_resource/utils/validate_submission.py	Official format checker (stdlib only)	Run it on your outputs before every upload. It checks format, not score.
dataset/train/train_source1.tsv	2,206,821 Source 1 records (US, India). 210 MB	Training reference records. Build blocking + features on these.
dataset/train/train_source2.tsv	5,034,616 Source 2 records. 489 MB	Training candidates (side A).
dataset/train/train_source3.tsv	5,285,603 Source 3 records. 504 MB	Training candidates (side B).
dataset/train/train_ground_truth.tsv	Correct answers for train: S1 id -> list of S2/S3 ids	Labels. Used to (a) measure blocking recall, (b) label pairs for the classifier, (c) score your validation F0.5.
dataset/test/test_source1.tsv	1,732,544 Source 1 records (US, India, France). 175 MB	You must output exactly one row for every ID in this file.
dataset/test/test_source2.tsv	4,887,273 records. 509 MB	Candidates to search for the test predictions.
dataset/test/test_source3.tsv	5,082,316 records. 506 MB	Candidates to search for the test predictions.
student_resource/.DS_Store	macOS junk	Ignore.

Columns in every source file (tab-separated):

Column	Meaning
entity_id	Unique ID. Prefix S1-, S2-, S3- tells the source.
business_name	Noisy name: abbreviations, legal suffixes, typos, transliteration, websites, trade names.

Column	Meaning
business_address	Noisy address: can be empty, partial, reordered, abbreviated, landmark based.
country	US / India in train; US / India / France in test. Treat as open set of strings.

Always read with **sep='\\t'**. Also use **dtype=str, keep_default_na=False, quoting=3** so empty addresses stay as empty strings and quotes inside names do not break parsing.

3. What the data looks like

Real training example: S1-965667 has 5 true matches.

ID	Name	Address	Why it matches
S1-965667	Maure Williams Colombier Inc	85 Wayne Avenue, Ticonderoga, NY	reference
S2-681193310	Maure Wilblims Colombier Inc	(empty)	name with typo
S2-743505751	Maure Williams Colombier	(empty)	legal suffix dropped
S3-860443364	Maure Williams Inc Center	(empty)	words reordered / added
S3-775321672	Dréxkor	85 Wanye Avenue, Ticonderoga Township, New York	totally different name (trade name) but same address with typos
S3-11291185	maurewilliamscolombier.com	Wayne Ave, Ticonderoga Township, New York	website-style name; address abbreviated

Lesson: a match can be proven by the name alone, the address alone, or both. Your blocking must search on **both** name and address, and your model must handle missing fields.

Statistics (computed from the files)

Number of true matches per S1 entity (train)	Share of S1 entities
0 (singleton)	5.6%
1	5.4%
2	17.0%
3	24.1%
4	21.9%
5	14.6%
6 or more	11.4%

Country	train S1	test S1	test S2
US	1,323,633	663,106	1,871,330
India	883,188	809,986	2,312,565
France	0 (never seen)	259,452 (15%)	703,378

France is only in test. About 15% of the test S1 records are French, with accented street names like 'Boulevard du Président Franklin Roosevelt'. Do not build features that only work for US/India (no one-hot of country, no US-only ZIP rules as the only signal). Use language-agnostic features: character n-grams, digit tokens, accent-stripped text, multilingual embeddings.

Noise patterns you must handle

- Names: Corp/Corporation, Pvt/Private, Ltd/Limited, Inc/LLC placed anywhere, '&' vs 'and', word swaps, typos, website domains (abc.com), trade/DBA names, leading junk like '--' or '<<'.
- Names in Devanagari (e.g. 'Ram Marketing Private Limited' written in Hindi script).
- Addresses: Rd/Road, St/Street, Ave/Avenue, state abbreviations (NY/New York), missing PIN/ZIP/state, components reordered ('IA, Iowa City, 1064 Newton Rd'), landmarks ('Near SBI ATM'), UPPERCASE vs lowercase, typos ('Wanye', 'Township').

4. How you are scored

For every Source 1 entity in the evaluation set: Precision = correct predicted / all predicted; Recall = correct predicted / all true. Then $F0.5 = 1.25 \times P \times R / (0.25 \times P + R)$. The final score is the **average over all S1 entities** (macro average).

Situation for one S1 entity	Score
True list empty, you predict empty	1.0
True list empty, you predict anything	0.0
True list non-empty, you predict empty	0.0
Predict [A,B,C], truth [A,C]: P=0.667, R=1.0	0.714
Predict [A], truth [A,C]: P=1.0, R=0.5	0.833

Look at the last two rows: predicting **fewer but correct** matches (0.833) beats predicting **all plus one wrong** (0.714). So when in doubt, leave it out. Your threshold should be fairly high.

Write your own scorer (use it on a held-out validation split of train):

```
def f05(pred: set, true: set) -> float:
    if not true and not pred: return 1.0
    if not true or not pred: return 0.0
    tp = len(pred & true)
    if tp == 0: return 0.0
    p, r = tp / len(pred), tp / len(true)
    return 1.25 * p * r / (0.25 * p + r)

def macro_f05(pred_dict, true_dict): # dict: s1_id -> set of ids
    return sum(f05(pred_dict.get(k, set()), v) for k, v in true_dict.items()) / len(true_dict)
```

5. Project folder you should create

Create a working folder next to the data. The layout below already matches the final zip, so you do not have to reorganise on Day 3.

```
D:\student_resource\          <- given (data lives here, do not modify)
  dataset\train\*.tsv
  dataset\test\*.tsv
  utils\validate_submission.py
  Documentation_template.md

D:\ml_challenge\              <- YOUR work folder (create it)
  output\
    matching_results.tsv      <- upload this to the portal
    candidate_pairs.tsv
  code\business_entity_resolution\
    src\
      config.py               <- paths, K values, thresholds
      io_utils.py             <- load TSVs, write outputs
      normalize.py            <- text cleaning
      blocking.py             <- candidate generation
      features.py             <- pair features
      train.py                <- train LightGBM on train split
      evaluate.py             <- F0.5 scorer + threshold search
      predict.py              <- run on test, write outputs
      run_all.py              <- end-to-end entry point
    README.md
    requirements.txt
  cache\
  Documentation_template.md    <- parquet files, models (NOT zipped)
                               <- copy + fill in at the end
```

Environment setup (PowerShell):

```
cd D:\ml_challenge
python -m venv .venv
.\venv\Scripts\Activate.ps1
pip install pandas pyarrow numpy scikit-learn scipy rapidfuzz unidecode lightgbm jellyfish
pip install sparse_dot_topn      # fast sparse top-K cosine (optional but useful)
pip install sentence-transformers faiss-cpu  # optional, for embedding blocking (Day 3)
pip install indic-transliteration  # Devanagari -> Latin
pip freeze > code\business_entity_resolution\requirements.txt
```

Memory: the full test set is ~11.7M records. Work on a **sample first** (e.g. 100k S1 entities plus all their ground-truth matches plus random S2/S3 noise), then scale. Convert TSVs to Parquet once to make reloads fast.

6. Step-by-step pipeline

Step 0: Load data and cache as Parquet (file: io_utils.py)

Reads: all 7 TSV files. **Writes:** cache/*.parquet.

```
import pandas as pd
DATA = r"D:\student_resource\dataset"

def load(split, n):
    return pd.read_csv(f"{DATA}\\{split}\\{split}_source{n}.tsv", sep="\t", dtype=str,
                      keep_default_na=False, quoting=3)

for split in ["train", "test"]:
    for n in [1, 2, 3]:
        load(split, n).to_parquet(f"cache/{split}_s{n}.parquet")

gt = pd.read_csv(f"{DATA}\\train\\train_ground_truth.tsv", sep="\t", dtype=str,
                 keep_default_na=False, quoting=3)
gt["matches"] = gt["matched_entity_ids"].apply(lambda s: set(s.split(",")) if s else set())
```

Step 1: Exploratory analysis (notebook, Day 1 morning)

Reads: train_source1/2/3, train_ground_truth. **Goal:** understand what makes a pair match.

- Print 50 random S1 entities with all their matched S2/S3 records side by side (like the table in Section 3).
- Count empty names / empty addresses per source.
- Check: how often does a true match share the house number? the ZIP/PIN? the city? at least one name token?
- Look at singletons: what do they look like? Are there near-duplicates that are NOT matches (hard negatives)?
- Check whether S2 and S3 behave differently (S3 seems to have more website names and trade names).

Write findings down; they go into Section 2.1 of the documentation.

Step 2: Normalisation (file: normalize.py)

Reads: cached source parquet. **Writes:** extra columns name_norm, name_core, addr_norm, numbers, postcode, city_tokens.

- Lowercase, unidecode (removes accents: 'Président' -> 'president'), transliterate Devanagari to Latin.
- Remove junk punctuation, turn '&' into 'and', collapse whitespace.
- Website names: 'maurewilliamscolombier.com' -> strip www. and .com/.in/.fr -> 'maurewilliamscolombier'. Also keep a version with all spaces removed for every name, so 'maure williams colombier' and the domain can be compared directly.
- Expand abbreviations with a dictionary: pvt->private, ltd->limited, corp->corporation, co->company, rd->road, st->street, ave->avenue, blvd->boulevard, ...
- name_core = name with legal suffixes removed (inc, llc, ltd, limited, private, pvt, corp, company, sarl, sas, llp, ...). Legal suffixes are often inconsistent so they should not dominate similarity.
- From the address extract: all digit tokens (house number, unit), postcode (5-digit US ZIP, 6-digit Indian PIN, 5-digit French code), and a city/locality token set.

```
import re
from unidecode import unidecode
LEGAL = {"inc", "llc", "ltd", "limited", "private", "pvt", "corp", "corporation", "co", "company",
        "llp", "plc", "sarl", "sas", "sa", "gmbh", "the"}
ABBR = {"rd": "road", "st": "street", "ave": "avenue", "blvd": "boulevard", "dr": "drive",
        "ln": "lane", "ct": "court", "pvt": "private", "ltd": "limited", "corp": "corporation", "&": "and"}

def norm(s: str) -> str:
    s = unidecode(s).lower()
    s = re.sub(r"https?://|www\.|", "", s)
    s = re.sub(r"\.(com|in|fr|net|org|co)\b", " ", s)
    s = re.sub(r"^[a-z0-9& ]+", " ", s)
```

```

return " ".join(ABBR.get(t, t) for t in s.split())

def core(name_norm: str) -> str:
    return " ".join(t for t in name_norm.split() if t not in LEGAL)

```

Step 3: Blocking / candidate generation (file: blocking.py)

Reads: normalised S1, S2, S3. **Writes:** a table of (s1_id, cand_id, which_blocker, rank, score). This table is what finally becomes **candidate_pairs.tsv**.

Use several blockers and take the **union**, because each catches different kinds of matches:

Blocker	How	Catches
A. Name char n-gram TF-IDF	TfidfVectorizer(analyzer='char_wb', ngram_range=(3,3)) on name_core; top-K cosine neighbours for each S1 (sparse_dot_topn or chunked matrix multiply). K about 10-20 per source.	Typos, reordering, suffix changes, website names
B. Address char n-gram TF-IDF	Same on addr_norm (only for non-empty addresses).	Trade names / DBA with same address (the 'Dréxxor' case)
C. Exact keys	Join on (house number + first street token + postcode) or (postcode + first name token), and on the space-free name.	Cheap, very precise hits
D. Embeddings (optional, Day 3)	Small multilingual sentence model (e.g. multilingual-e5-small, MIT) on 'name address'; FAISS top-K.	Transliteration, French, semantic variants

- **Partition by country** before searching (only compare US with US etc.). This is allowed: you are grouping by whatever labels exist, not hard-coding the list. France will simply be its own group.
- Search S2 and S3 separately (top-K from each) so one source cannot crowd out the other.
- Process S1 in chunks of 10k-50k rows to control memory.

Measure blocking recall on train (must do):

```

# cands: DataFrame(s1_id, cand_id)  gt: s1_id -> set of true ids
true_pairs = {(s, c) for s, cs in gt_dict.items() for c in cs}
cand_pairs = set(zip(cands.s1_id, cands.cand_id))
recall = len(true_pairs & cand_pairs) / len(true_pairs)
print("blocking recall", recall, "avg cands per S1", len(cand_pairs) / len(gt_dict))

```

Targets: blocking recall **above 95%** with roughly **20-60 candidates per S1**. Whatever is lost here can never be recovered by the model. Tune K and the blocker mix until recall is high.

Step 4: Pair features (file: features.py)

Reads: candidate table + normalised records. **Writes:** feature matrix (one row per candidate pair).

Group	Features (use rapidfuzz for speed)
Name	ratio, partial_ratio, token_sort_ratio, token_set_ratio, Jaro-Winkler on name_norm and name_core; TF-IDF cosine (from blocking); space-free name equality and containment (domain names); first-token equal; number of shared tokens; length difference; phonetic match (jellyfish metaphone).
Address	token_set_ratio on addr_norm; house number equal; any digit token overlap; postcode equal / missing; city token overlap; street token overlap; TF-IDF address cosine.
Missingness	S1 address empty, candidate address empty, candidate name empty (these change which features are reliable; let the model learn it).
Context	candidate source (S2 or S3), rank within this S1's candidates, score minus best score for this S1, number of candidates for this S1, how many different S1s picked this same candidate (a candidate that is the nearest neighbour of many S1s is less certain).
Country	Do NOT one-hot. At most use country-agnostic things like 'script is non-Latin'.

Step 5: Train the classifier (file: train.py)

Reads: train features + labels from train_ground_truth.tsv. **Writes:** cache/model.txt.

- Label = 1 if cand_id is in the ground-truth set of that s1_id, else 0.
- Split by **S1 entity** (GroupKFold or simple 80/20 on s1_id). Never split by row, or pairs of the same entity leak between train and validation.
- Model: LightGBM binary classifier (fast, handles missing values, strong on tabular features). Start with num_leaves=63, learning_rate=0.05, 500-1500 rounds with early stopping.
- You do not need all 2.2M train S1 entities; 300k-500k S1 entities of pairs is usually plenty.

```
import lightgbm as lgb
model = lgb.LGBMClassifier(n_estimators=2000, learning_rate=0.05, num_leaves=63,
                           subsample=0.8, colsample_bytree=0.8)
model.fit(X_tr, y_tr, eval_set=[(X_va, y_va)],
          callbacks=[lgb.early_stopping(100)])
va["prob"] = model.predict_proba(X_va)[:, 1]
```

Step 6: Decision rule and threshold (file: evaluate.py)

Reads: validation probabilities. **Writes:** chosen threshold in config.py.

- For each S1: keep candidates with prob $\geq t$. Grid search t from 0.3 to 0.95 and pick the value with the best **macro F0.5** (Section 4 scorer), not the best accuracy or AUC.
- Try a separate singleton rule: if the highest prob for this S1 is below t_0 , predict empty.
- Optional: per-source thresholds (S2 vs S3), or transitivity checks (if two candidates are clearly the same business as each other, they strengthen each other).

Step 7: Inference on test (file: predict.py)

Reads: test_source1/2/3. **Writes:** output/candidate_pairs.tsv and output/matching_results.tsv.

Run exactly the same normalise, block, feature, predict, threshold steps on the test files, in chunks of S1 rows, using the saved model and threshold.

7. Producing the two output files

File	Header (tab separated)	Content
output/matching_results.tsv	source1_entity_id matched_entity_ids	Final matches. Scored on the leaderboard. Upload this one.
output/candidate_pairs.tsv	source1_entity_id candidate_entity_ids	The exact candidate list your model scored (after the last blocking/filter stage). Not scored, but required in the zip.

- Exactly one row for every entity_id in test_source1.tsv (1,732,544 rows + header), including France and singletons.
- Empty second column for no matches (the line is 'S1-xxx' followed by a tab and nothing).
- IDs comma-separated, no spaces, no quotes, no duplicates, only S2-/S3- IDs.
- Every matched ID must also appear in that S1's candidate list.

```
def write_output(s1_ids, id_lists, path, col):
    # s1_ids: list of all test S1 ids; id_lists: dict s1_id -> list of ids
    with open(path, "w", encoding="utf-8", newline="\n") as f:
        f.write(f"source1_entity_id\t{col}\n")
        for s in s1_ids:
            ids = list(dict.fromkeys(id_lists.get(s, []))) # dedupe, keep order
            f.write(s + "\t" + ",".join(ids) + "\n")

s1_ids = test_s1["entity_id"].tolist()
write_output(s1_ids, cands_by_s1, "output/candidate_pairs.tsv", "candidate_entity_ids")
write_output(s1_ids, match_by_s1, "output/matching_results.tsv", "matched_entity_ids")
```

8. Validating and uploading

Run the official validator from the student_resource folder before **every** upload:

```
cd D:\student_resource
python utils\validate_submission.py `
    --matching D:\ml_challenge\output\matching_results.tsv `
    --candidate D:\ml_challenge\output\candidate_pairs.tsv `
    --test-dir dataset\test
# optional, heavier (few GB RAM): add --check-ids to confirm every ID exists in test
```

- PASS (exit 0) means safe to upload. Otherwise fix the numbered issues.
- Upload only **matching_results.tsv** on the portal. You should see status SCORED and an F0.5.
- You get 5 uploads per day (15 in total). Do not waste them on format errors.
- Keep a log: date/time, git commit or folder copy, validation F0.5, leaderboard F0.5, what changed. The guidelines require you to keep version history of submissions.

The public leaderboard uses only part of the test set; the final rank uses the private part. Trust your own validation F0.5 more than small public leaderboard jumps, to avoid overfitting to the public split.

9. Final submission zip

```
<team_name>_submission.zip
|-- output/
|   |-- matching_results.tsv      (same file as your best leaderboard upload)
|   |-- candidate_pairs.tsv
|-- code/
|   |-- business_entity_resolution/
|   |   |-- src/                  (all source code, commented functions)
|   |   |-- README.md             (exact commands: data -> blocking -> matching -> output)
|   |   |-- requirements.txt       (pinned versions, from pip freeze)
|   |-- Documentation_template.md (filled in; .pdf export also accepted; do not rename)
```

- Do NOT include the dataset or large cache files in the zip.
- README.md must let someone regenerate both output files from the raw data using only this folder. Put the data path in config.py or as a command-line argument.

- Documentation must cover: methodology, blocking strategy (keys, number of pairs, blocking recall), model and features, threshold choice, validation F0.5, typical false positives and false negatives.
- The guidelines also mention a 1-2 page write-up of approach, models, experiments and conclusion: the filled template covers this.

10. Rules that get you rejected or disqualified

Rule	Consequence
Missing any test S1 entity, or duplicate S1 rows	Submission rejected
Duplicate IDs inside a list, S1 IDs inside a list	Submission rejected
Wrong header names / not tab-separated	Submission rejected
Using external data: geocoding APIs, business registries, web lookups, commercial ER services, internet data augmentation	Immediate disqualification
Final model not MIT / Apache 2.0, or larger than 8B parameters	Not allowed
Multiple accounts / simultaneous logins / plagiarism	Disqualification
Hard-coding country to {US, India} or dropping France rows	Missing rows = rejection; weak France score

Allowed: open-source libraries (pandas, scikit-learn, LightGBM, rapidfuzz), pretrained open models under MIT/Apache up to 8B params used only on the provided data, and your own hand-written abbreviation dictionaries.

11. 3-day schedule

When	Work	Deliverable
Day 1 (25 Sep) morning	Setup venv, cache Parquet, EDA on train, write scorer.	Notes for documentation; f05 scorer
Day 1 afternoon	Normalisation + name TF-IDF blocking on a 100k-S1 train sample; measure blocking recall.	Recall number
Day 1 evening	Baseline: match if name similarity > high threshold. Run on test, validate, upload.	First SCORED submission
Day 2 (26 Sep)	Add address + exact-key blockers; build pair features; train LightGBM; tune threshold on macro F0.5; full-scale test inference.	2-4 improved submissions
Day 3 (27 Sep) morning	Error analysis on validation (look at false merges first); embedding blocker for France/transliteration; per-source thresholds.	Best submission
Day 3 afternoon	Freeze code, write README + requirements, fill Documentation_template.md, build zip, re-run pipeline from scratch once to check reproducibility.	Final zip before 11:59 PM IST

12. Final checklist

- [] Read both PDFs and student_resource/README.md
- [] Used data from D:\student_resource (not the __MACOSX folder)
- [] All TSVs read with sep='\t', dtype=str, keep_default_na=False
- [] Validation split made by S1 entity; own macro F0.5 scorer works
- [] Blocking recall measured on train (target above 95%)
- [] Threshold chosen by maximising macro F0.5 on validation
- [] France records processed (no country hard-coding)
- [] matching_results.tsv has 1,732,544 data rows + header
- [] candidate_pairs.tsv has the same rows; matches are a subset of candidates

[] validate_submission.py prints PASS

[] Submission log kept (time, version, scores)

[] Zip: output/, code/business_entity_resolution/{src, README.md, requirements.txt}, Documentation_template.md

[] No external lookups anywhere in the code; all models MIT/Apache and 8B params or smaller